

Ibrahim Ferizarizqi Permana 18224120

## **Tugas Sistem Multimedia**

### **Digital Watermarking Menggunakan Metode LSB**

### **dengan Evaluasi Kompresi JPEG**

Foto awal :



#### **Tujuan Tugas :**

- Mengimplementasikan teknik watermarking dengan metode LSB pada foto wajah pribadi.
- Menyisipkan watermark biner ke dalam gambar menggunakan bit paling tidak signifikan (LSB) pada channel warna biru.
- Mengompres gambar hasil watermark menggunakan standar kompresi JPEG pada beberapa nilai Quality Factor (QF).
- Mengekstrak kembali watermark dari setiap versi gambar yang telah dikompres.

- Mengevaluasi kualitas gambar dan ketahanan watermark menggunakan metrik PSNR, BER, dan NCC.
- Menentukan nilai QF minimum yang membuat watermark tidak dapat diekstrak kembali.

## Dasar Teori

### LSB (Least Significant Bit) Watermarking

LSB watermarking adalah salah satu metode paling sederhana dalam steganografi domain spasial. Setiap piksel gambar digital direpresentasikan oleh nilai integer 8-bit (0–255). Bit paling tidak signifikan (bit ke-0) memberikan kontribusi nilai hanya sebesar 1 dari 256, sehingga perubahannya hampir tidak terdeteksi secara visual.

Proses embedding dilakukan dengan mengganti LSB pada channel tertentu (dalam implementasi ini digunakan channel Biru / Blue) dengan bit watermark secara berurutan. Secara matematis, proses embedding dapat ditulis sebagai:

$$P'[i] = (P[i] \text{ AND } 0xFE) \text{ OR } W[i]$$

di mana  $P[i]$  adalah nilai piksel asli,  $W[i]$  adalah bit watermark (0 atau 1), dan  $P'[i]$  adalah nilai piksel hasil embedding.

Proses ekstraksi dilakukan dengan mengambil LSB dari setiap piksel pada posisi yang telah didefinisikan:

$$W'[i] = P'[i] \text{ AND } 0x01$$

### Kompresi JPEG

JPEG (Joint Photographic Experts Group) adalah standar kompresi lossy yang umum digunakan pada gambar digital. Kompresi JPEG menggunakan transformasi DCT

(Discrete Cosine Transform), kuantisasi, dan pengkodean Huffman. Proses kuantisasi menyebabkan hilangnya informasi frekuensi tinggi, termasuk perubahan bit LSB yang disisipkan oleh metode watermarking spasial.

Quality Factor (QF) adalah parameter yang mengontrol tingkat kompresi JPEG, dengan rentang nilai 1 (kualitas terburuk, kompresi tertinggi) hingga 95 (kualitas terbaik, kompresi rendah). Semakin rendah nilai QF, semakin besar distorsi yang terjadi pada gambar, dan semakin besar kemungkinan watermark rusak.

## **Metrik Evaluasi**

Tiga metrik digunakan untuk mengevaluasi performa watermarking:

### **a. PSNR (Peak Signal-to-Noise Ratio)**

PSNR mengukur kualitas gambar terkompresi dibandingkan gambar asli, dalam satuan desibel (dB). Nilai PSNR yang lebih tinggi menunjukkan kualitas gambar yang lebih baik (distorsi lebih kecil). Secara umum,  $\text{PSNR} > 30 \text{ dB}$  dianggap masih dapat diterima secara visual.

$$\text{PSNR} = 10 \times \log_{10}(255^2 / \text{MSE})$$

### **b. BER (Bit Error Rate)**

BER mengukur proporsi bit watermark yang salah setelah ekstraksi dibandingkan watermark asli. Nilai BER berkisar 0 hingga 1, di mana  $\text{BER} = 0$  berarti semua bit benar, dan  $\text{BER} = 1$  berarti semua bit salah. Dalam implementasi ini, watermark dianggap tidak dapat diekstrak apabila  $\text{BER} > 0.3$ .

$$\text{BER} = (\text{jumlah bit salah}) / (\text{total bit watermark})$$

### **c. NCC (Normalized Cross-Correlation)**

NCC mengukur kemiripan antara watermark asli dan watermark yang diekstrak. Nilai NCC berkisar dari -1 hingga 1, di mana  $NCC = 1$  berarti identik sempurna. Watermark dianggap tidak dapat diekstrak apabila  $NCC < 0.5$ .

## Langkah Implementasi

Implementasi dilakukan menggunakan Google Colaboratory (Colab) dengan notebook Python. Seluruh langkah dijalankan secara berurutan sesuai struktur notebook yang telah dibuat. Berikut adalah penjelasan setiap tahapan implementasi.

### Install dan Import Library

Langkah pertama adalah memastikan seluruh library yang dibutuhkan telah tersedia dan diimport ke dalam environment. Pada Google Colab, library NumPy, Pillow (PIL), dan Matplotlib sudah tersedia secara default sehingga tidak memerlukan instalasi tambahan.

Library yang digunakan adalah sebagai berikut:



```
[1] ✓ Os
# Library sudah tersedia di Colab, tidak perlu install tambahan
import numpy as np
from PIL import Image, ImageDraw, ImageFont
import io, os
import matplotlib.pyplot as plt
from IPython.display import display

print('✓ Library siap')
```

✓ Library siap

- numpy (np) : digunakan untuk manipulasi array multidimensi, operasi matematika, dan perhitungan metrik.
- PIL (Pillow) : digunakan untuk membuka, menyimpan, dan memanipulasi gambar dalam berbagai format.
- io : digunakan untuk membuat buffer memori sehingga proses kompresi JPEG dapat dilakukan di RAM tanpa menyimpan file secara fisik.

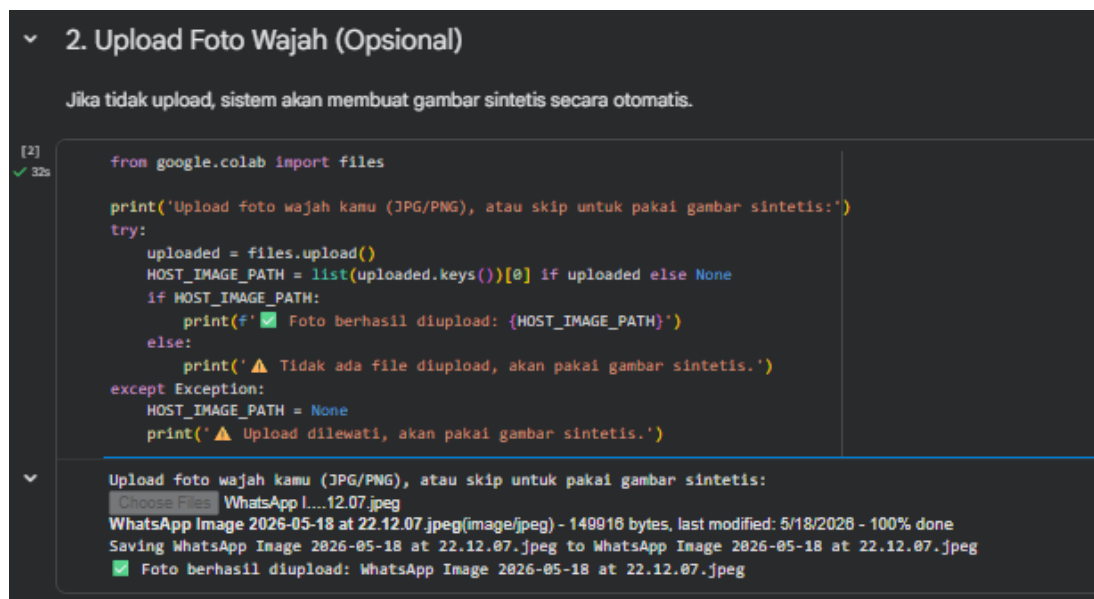
- os : digunakan untuk manajemen direktori output.
- matplotlib.pyplot : digunakan untuk membuat plot dan visualisasi hasil evaluasi.
- IPython.display : digunakan untuk menampilkan gambar langsung di dalam notebook Colab.

Setelah sel dijalankan, output konfirmasi '✅ Library siap' akan muncul, menandakan seluruh library berhasil diimport tanpa error.

## Upload Foto Wajah

Pada langkah ini, pengguna dapat mengunggah foto wajah pribadi yang akan digunakan sebagai host image (gambar pembawa watermark). Apabila pengguna tidak mengunggah foto, sistem secara otomatis akan membuat gambar sintetis berukuran 256 x 256 piksel sebagai pengganti.

Proses upload dilakukan menggunakan modul files dari google.colab:



```

2. Upload Foto Wajah (Opsional)

Jika tidak upload, sistem akan membuat gambar sintetis secara otomatis.

[2]
✓ 32s
from google.colab import files

print('Upload foto wajah kamu (JPG/PNG), atau skip untuk pakai gambar sintetis:')
try:
    uploaded = files.upload()
    HOST_IMAGE_PATH = list(uploaded.keys())[0] if uploaded else None
    if HOST_IMAGE_PATH:
        print(f'✅ Foto berhasil diupload: {HOST_IMAGE_PATH}')
    else:
        print('⚠️ Tidak ada file diupload, akan pakai gambar sintetis.')
except Exception:
    HOST_IMAGE_PATH = None
    print('⚠️ Upload dilewati, akan pakai gambar sintetis.')

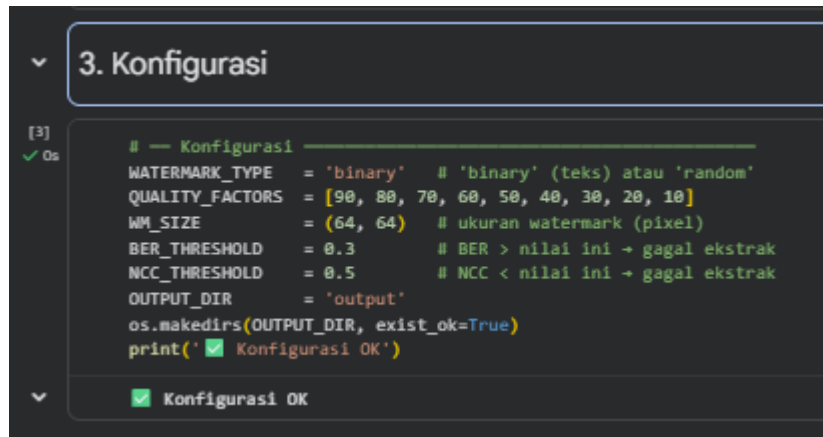
Upload foto wajah kamu (JPG/PNG), atau skip untuk pakai gambar sintetis:
Choose Files WhatsApp Image 2026-05-18 at 22.12.07.jpeg
WhatsApp Image 2026-05-18 at 22.12.07.jpeg(image/jpeg) - 149916 bytes, last modified: 5/18/2026 - 100% done
Saving WhatsApp Image 2026-05-18 at 22.12.07.jpeg to WhatsApp Image 2026-05-18 at 22.12.07.jpeg
✅ Foto berhasil diupload: WhatsApp Image 2026-05-18 at 22.12.07.jpeg
  
```

Pada percobaan ini, digunakan foto wajah pribadi berformat JPEG dengan nama file 'WhatsApp Image 2026-05-18 at 22.12.07.jpeg' berukuran sekitar 140 KB. Setelah berhasil diunggah, sistem menampilkan konfirmasi nama file dan ukurannya.

*Proses upload foto wajah dan output konfirmasi keberhasilan upload*

## Konfigurasi Parameter

Sebelum menjalankan pipeline utama, seluruh parameter eksperimen didefinisikan dalam satu sel konfigurasi. Hal ini bertujuan agar seluruh nilai yang digunakan dapat dengan mudah dimodifikasi tanpa mengubah kode di sel-sel lain.



```
[3]
✓ Os
# -- Konfigurasi --
WATERMARK_TYPE = 'binary' # 'binary' (teks) atau 'random'
QUALITY_FACTORS = [90, 80, 70, 60, 50, 40, 30, 20, 10]
WM_SIZE = (64, 64) # ukuran watermark (pixel)
BER_THRESHOLD = 0.3 # BER > nilai ini → gagal ekstrak
NCC_THRESHOLD = 0.5 # NCC < nilai ini → gagal ekstrak
OUTPUT_DIR = 'output'
os.makedirs(OUTPUT_DIR, exist_ok=True)
print('✅ Konfigurasi OK')
```

✅ Konfigurasi OK

Penjelasan setiap parameter konfigurasi:

- `WATERMARK_TYPE = 'binary'`: watermark yang digunakan adalah gambar biner berupa teks 'ITB' berukuran 64 x 64 piksel. Opsi alternatif adalah 'random' untuk watermark acak.
- `QUALITY_FACTORS`: daftar nilai QF JPEG yang akan diuji, dimulai dari kualitas tinggi (QF=90) hingga kualitas sangat rendah (QF=10), dengan interval 10.
- `WM_SIZE = (64, 64)`: ukuran watermark 64 x 64 piksel, menghasilkan 4.096 bit yang akan disisipkan ke dalam gambar.
- `BER_THRESHOLD = 0.3`: jika BER hasil ekstraksi melebihi 0.3, watermark dianggap tidak dapat diekstrak.
- `NCC_THRESHOLD = 0.5`: jika NCC hasil ekstraksi di bawah 0.5, watermark dianggap tidak dapat diekstrak.
- `OUTPUT_DIR = 'output'`: semua file hasil (gambar kompresi, watermark terekstrak, plot) disimpan di folder 'output'.

## Pendefinisian Fungsi-Fungsi Utama

Seluruh fungsi inti yang digunakan dalam pipeline watermarking didefinisikan pada langkah ini. Terdapat lima kelompok fungsi utama, yaitu generate watermark, embed LSB, kompresi JPEG, ekstrak LSB, dan metrik evaluasi.

#### **a. Generate Watermark Biner (generate\_binary\_watermark)**

Fungsi ini membuat watermark berupa teks 'ITB' yang digambar pada kanvas hitam berukuran 64 x 64 piksel, kemudian dibinarisasi sehingga nilai piksel hanya berupa 0 (hitam) atau 1 (putih).

```
def generate_binary_watermark(size=(64,64), text='WM'):  
    img = Image.new('L', size, color=0)  
    draw = ImageDraw.Draw(img)  
    try:  
        font = ImageFont.load_default()  
    except:  
        font = None  
    draw.text((4, 20), text, fill=255, font=font)  
    wm = np.array(img)  
    return (wm > 128).astype(np.uint8)
```

#### **b. Embed LSB (embed\_lsb)**

Fungsi ini menyisipkan watermark ke dalam gambar host dengan mengganti LSB pada channel Biru (index 2) secara berurutan piksel demi piksel. Operasi AND dengan 0xFE membersihkan bit LSB yang lama, kemudian OR dengan bit watermark menyisipkan nilai baru.

```
def embed_lsb(image_array, watermark):  
    img = image_array.copy()  
    wm_flat = watermark.flatten()  
    h, w, _ = img.shape  
    blue = img[:, :, 2].flatten()  
    blue[:len(wm_flat)] = (blue[:len(wm_flat)] & 0xFE) | wm_flat  
    img[:, :, 2] = blue.reshape(h, w)  
    return img
```

#### **c. Kompresi JPEG (compress\_jpeg)**

Fungsi ini mengompres gambar numpy array ke format JPEG pada quality factor tertentu menggunakan buffer memori (io.BytesIO), kemudian mendecode kembali hasilnya menjadi numpy array. Dengan demikian proses dilakukan sepenuhnya di memori tanpa menyimpan file sementara.

```
def compress_jpeg(image_array, quality):
    img_pil = Image.fromarray(image_array.astype(np.uint8))
    buf = io.BytesIO()
    img_pil.save(buf, format='JPEG', quality=quality)
    buf.seek(0)
    return np.array(Image.open(buf))
```

#### **d. Ekstrak LSB (extract\_lsb)**

Fungsi ini mengekstrak watermark dari gambar dengan mengambil LSB channel Biru pada posisi yang sama dengan saat embedding. Hasilnya kemudian dibentuk ulang menjadi array 2D berukuran watermark\_shape.

```
def extract_lsb(image_array, watermark_shape):
    n_bits = watermark_shape[0] * watermark_shape[1]
    blue = image_array[:, :, 2].flatten()
    return (blue[:n_bits] & 1).reshape(watermark_shape).astype(np.uint8)
```

#### **e. Metrik Evaluasi (psnr, ber, ncc)**

Ketiga fungsi metrik mengimplementasikan rumus PSNR, BER, dan NCC seperti yang dijelaskan pada bagian dasar teori. Fungsi psnr menangani kasus MSE=0 (gambar identik) dengan mengembalikan nilai tak terhingga.



```

# -- 4a. Generate Watermark -----

def generate_binary_watermark(size=(64,64), text='WM'):
    img = Image.new('L', size, color=0)
    draw = ImageDraw.Draw(img)
    try:
        font = ImageFont.load_default()
    except:
        font = None
    draw.text((4, 28), text, fill=255, font=font)
    wm = np.array(img)
    return (wm > 128).astype(np.uint8)

def generate_random_watermark(size=(64,64), seed=42):
    rng = np.random.default_rng(seed)
    return rng.integers(0, 2, size=size, dtype=np.uint8)

# -- 4b. Embed LSB -----

def embed_lsb(image_array, watermark):
    img = image_array.copy()
    wm_flat = watermark.flatten()
    h, w, _ = img.shape
    blue = img[:, :, 2].flatten()
    blue[:len(wm_flat)] = (blue[:len(wm_flat)] & 0xFE) | wm_flat
    img[:, :, 2] = blue.reshape(h, w)
    return img

# -- 4c. Kompres JPEG -----

def compress_jpeg(image_array, quality):
    img_pil = Image.fromarray(image_array.astype(np.uint8))
    buf = io.BytesIO()
    img_pil.save(buf, format='JPEG', quality=quality)
    buf.seek(0)
    return np.array(Image.open(buf))

# -- 4d. Ekstrak LSB -----

def extract_lsb(image_array, watermark_shape):
    n_bits = watermark_shape[0] * watermark_shape[1]
    blue = image_array[:, :, 2].flatten()
    return (blue[:n_bits] & 1).reshape(watermark_shape).astype(np.uint8)

# -- 4e. Metrik -----

def psnr(original, compressed):
    mse = np.mean((original.astype(np.float64) - compressed.astype(np.float64))**2)
    if mse == 0: return float('inf')
    return 10 * np.log10(255**2 / mse)

def ber(wm_orig, wm_ext):
    return np.mean(wm_orig.flatten() != wm_ext.flatten())

def ncc(wm_orig, wm_ext):
    a = wm_orig.astype(np.float64) - wm_orig.mean()
    b = wm_ext.astype(np.float64) - wm_ext.mean()
    denom = np.sqrt(np.sum(a**2) * np.sum(b**2))
    if denom == 0: return 0.0
    return float(np.sum(a*b) / denom)

print('✅ Fungsi siap')

```

*Definisi seluruh fungsi utama watermarking pada notebook*

## Load Foto Host dan Generate Watermark

Pada langkah ini, foto wajah yang telah diupload sebelumnya dibuka dan dikonversi ke format array RGB. Jika foto tidak tersedia, sistem akan menggunakan gambar sintetis. Selanjutnya, watermark biner dibuat menggunakan fungsi `generate_binary_watermark` dengan teks 'ITB'.

Foto host berhasil dimuat dengan dimensi 1280 x 959 piksel (3 channel RGB). Watermark biner berukuran 64 x 64 piksel berhasil dibuat, yang mengandung 4.096 bit data yang akan disisipkan ke dalam gambar host.

Hasil visualisasi menampilkan dua panel berdampingan:

- Panel kiri: Foto host asli (foto wajah pribadi)
- Panel kanan: Watermark asli berupa teks 'ITB' berwarna putih di atas latar belakang hitam

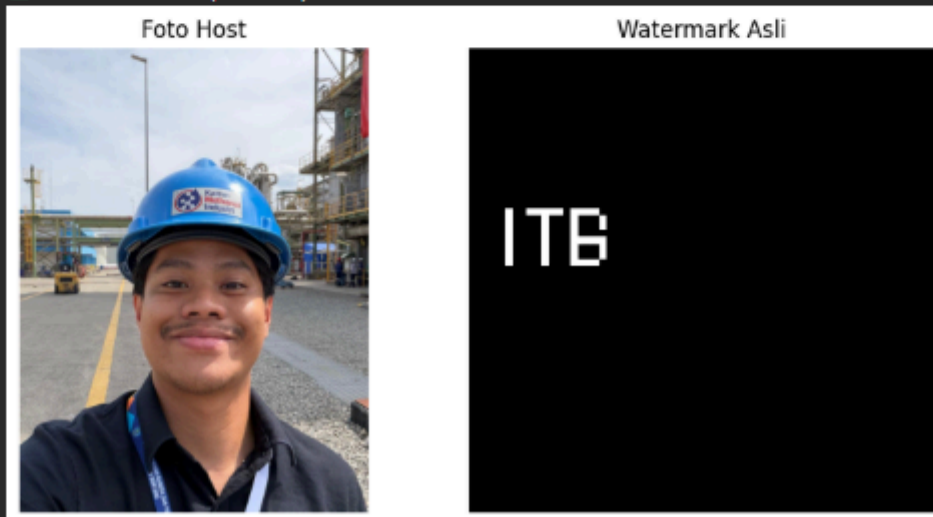
## 5. Load Foto & Buat Watermark

```
[5]
✓ Os
# Load foto host
if HOST_IMAGE_PATH and os.path.exists(HOST_IMAGE_PATH):
    host_img = np.array(Image.open(HOST_IMAGE_PATH).convert('RGB'))
    print(f'✓ Foto loaded: {host_img.shape}')
else:
    # Buat gambar sintetis 256x256
    h, w = 256, 256
    host_img = np.full((h, w, 3), [200, 170, 140], dtype=np.uint8)
    cx, cy = w//2, h//2
    for y in range(h):
        for x in range(w):
            if ((x-cx)/(w*0.35))**2 + ((y-cy)/(h*0.45))**2 < 1:
                host_img[y,x] = [220,185,155]
    host_img[cy-30:cy-20, cx-35:cx-15] = [50,30,20]
    host_img[cy-30:cy-20, cx+15:cx+35] = [50,30,20]
    host_img[cy+30:cy+38, cx-20:cx+20] = [160,80,80]
    print('✓ Gambar sintetis dibuat (256x256)')

# Buat watermark
if WATERMARK_TYPE == 'random':
    watermark = generate_random_watermark(WM_SIZE)
    print('✓ Watermark acak dibuat')
else:
    watermark = generate_binary_watermark(WM_SIZE, text='ITB')
    print('✓ Watermark biner (teks ITB) dibuat')

# Tampilkan
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
axes[0].imshow(host_img); axes[0].set_title('Foto Host'); axes[0].axis('off')
axes[1].imshow(watermark, cmap='gray', vmin=0, vmax=1)
axes[1].set_title('Watermark Asli'); axes[1].axis('off')
plt.tight_layout(); plt.show()
```

✓ Foto loaded: (1280, 959, 3)  
✓ Watermark biner (teks ITB) dibuat



## Embedding Watermark

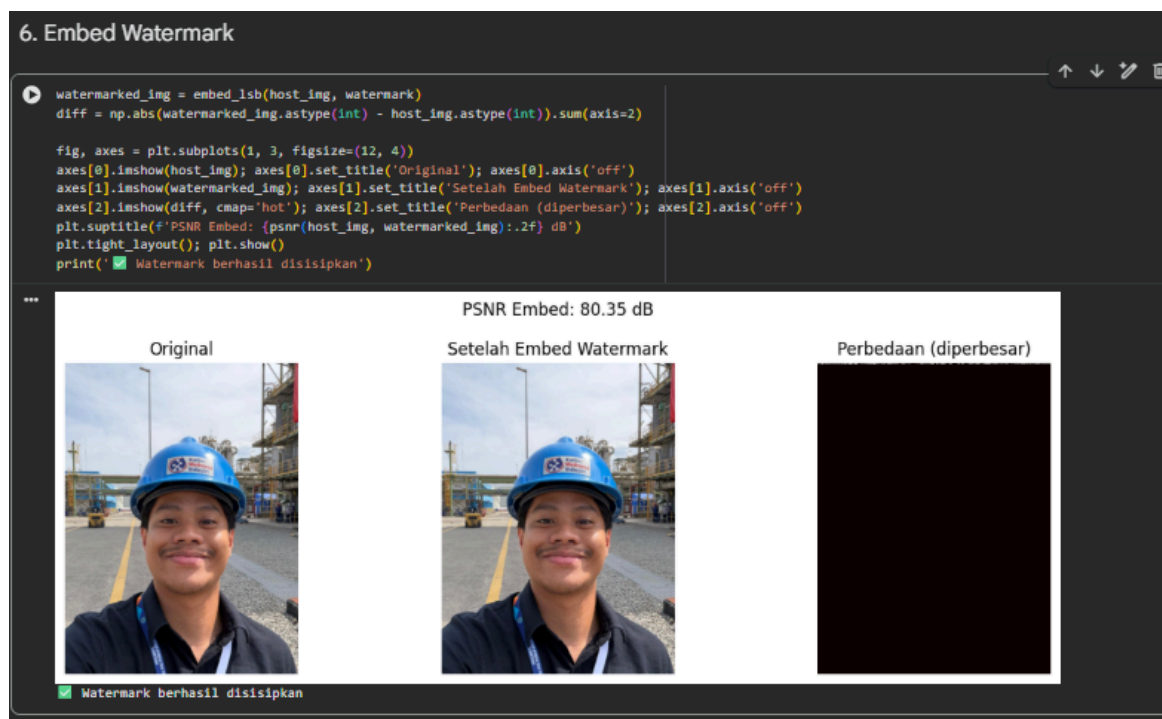
Watermark disisipkan ke dalam foto host menggunakan fungsi `embed_lsb`. Proses ini mengubah LSB pada channel Biru selama 4.096 piksel pertama (sesuai ukuran

watermark 64 x 64 bit). Perubahan yang terjadi sangat kecil sehingga tidak terlihat secara visual.

Hasil embedding divisualisasikan dalam tiga panel:

- Panel kiri (Original): Foto host asli sebelum embedding
- Panel tengah (Setelah Embed Watermark): Foto setelah watermark disisipkan — secara visual identik dengan foto asli
- Panel kanan (Perbedaan diperbesar): Tampilan peta perbedaan piksel menggunakan colormap 'hot'. Area gelap menandakan perubahan pada piksel yang disisipkan watermark

Nilai PSNR embedding yang diperoleh adalah 80.35 dB, jauh di atas threshold kualitas visual 30 dB. Hal ini membuktikan bahwa proses embedding LSB tidak mengubah kualitas gambar secara signifikan dan watermark bersifat tidak terlihat (invisible).



*Perbandingan foto asli, foto berisi watermark, dan peta perbedaan piksel (PSNR embed = 80.35 dB)*

## Evaluasi per Quality Factor JPEG

Pada tahap ini, gambar yang telah disisipi watermark dikompres menggunakan JPEG pada setiap nilai QF yang telah dikonfigurasi (90, 80, 70, 60, 50, 40, 30, 20, 10). Dari setiap hasil kompresi, watermark diekstrak kembali dan dievaluasi menggunakan metrik PSNR, BER, dan NCC.

Logika evaluasi untuk menentukan keberhasilan ekstraksi adalah sebagai berikut: watermark dianggap dapat diekstrak jika  $BER \leq 0.3$  DAN  $NCC \geq 0.5$ . Jika salah satu kondisi tidak terpenuhi, watermark dinyatakan tidak dapat diekstrak.

Tabel 1 berikut merangkum hasil evaluasi untuk seluruh nilai QF yang diuji:

**Tabel 1. Hasil Evaluasi Watermarking per Quality Factor JPEG**

7. Evaluasi per Quality Factor

```
[9]
✓ Os
results = []
print(f"{'QF':>5} {'PSNR (dB)':>12} {'BER':>8} {'NCC':>8} {'Status':>10}")
print('-' * 60)

for qf in QUALITY_FACTORS:
    compressed = compress_jpeg(watermarked_img, quality=qf)
    extracted_wm = extract_lsb(compressed, watermark.shape)
    p = psnr(watermarked_img, compressed)
    b = ber(watermark, extracted_wm)
    n = ncc(watermark, extracted_wm)
    can_extract = (b <= BER_THRESHOLD) and (n >= NCC_THRESHOLD)
    results.append({'qf': qf, 'psnr': p, 'ber': b, 'ncc': n,
                  'can_extract': can_extract, 'compressed': compressed,
                  'extracted_wm': extracted_wm})
    status = '✓ Dapat diekstrak' if can_extract else 'X GAGAL'
    p_str = f'{p:.2f}' if p != float('inf') else '∞'
    print(f"{'qf':>5} {'p_str':>12} {'b:>8.4f} {'n:>8.4f} {'status':>10}")

fail_qfs = [r['qf'] for r in results if not r['can_extract']]
if fail_qfs:
    threshold_qf = max(fail_qfs)
    print(f'\n🔴 Watermark TIDAK dapat diekstrak pada QF ≤ {threshold_qf}')
else:
    threshold_qf = None
    print('\n🟢 Watermark masih dapat diekstrak pada semua QF yang diuji.')
```

| QF | PSNR (dB) | BER    | NCC     | Status  |
|----|-----------|--------|---------|---------|
| 90 | 48.21     | 0.5254 | -0.0099 | X GAGAL |
| 80 | 45.61     | 0.5310 | 0.0079  | X GAGAL |
| 70 | 44.00     | 0.5415 | -0.0132 | X GAGAL |
| 60 | 38.52     | 0.5227 | 0.0334  | X GAGAL |
| 50 | 35.08     | 0.6328 | -0.0428 | X GAGAL |
| 40 | 34.68     | 0.3635 | 0.0387  | X GAGAL |
| 30 | 33.68     | 0.5925 | 0.0002  | X GAGAL |
| 20 | 31.75     | 0.5149 | 0.0112  | X GAGAL |
| 10 | 28.53     | 0.9187 | -0.0268 | X GAGAL |

🔴 Watermark TIDAK dapat diekstrak pada QF ≤ 90

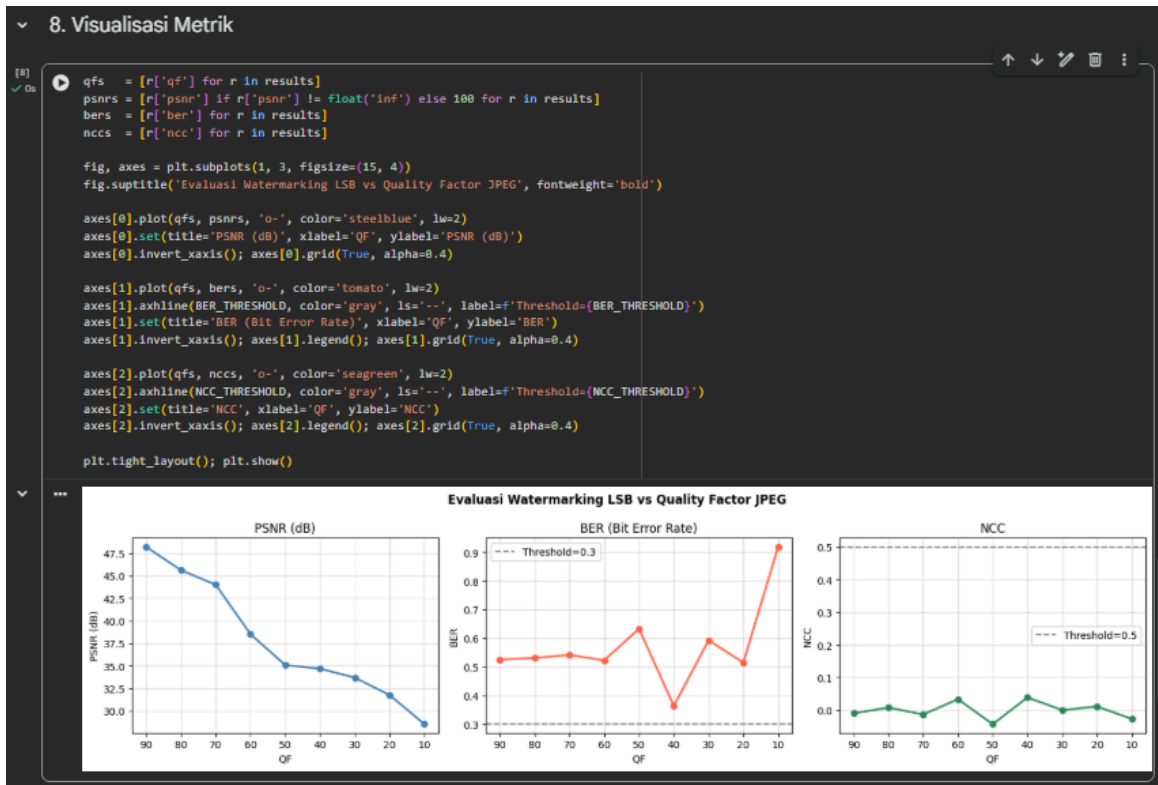
Dari tabel di atas, dapat dilihat bahwa pada semua nilai QF yang diuji (90 hingga 10), watermark tidak dapat diekstrak kembali. Hal ini ditunjukkan oleh nilai BER yang jauh melebihi threshold 0.3 (berkisar antara 0.36 hingga 0.92) serta nilai NCC yang sangat rendah dan tidak mencapai threshold 0.5 (berkisar antara -0.04 hingga 0.04). Hasil ini menunjukkan bahwa metode LSB sangat rentan bahkan terhadap kompresi JPEG dengan kualitas tertinggi sekalipun (QF=90).

## 5.8 Visualisasi Metrik Evaluasi

Untuk mempermudah interpretasi hasil evaluasi, ketiga metrik ditampilkan dalam bentuk grafik garis pada satu baris plot. Sumbu X pada setiap grafik menunjukkan nilai QF (urutan dibalik dari 90 ke 10, kiri ke kanan) sehingga arah grafik searah dengan meningkatnya tingkat kompresi.

Tiga grafik yang ditampilkan adalah:

- Grafik PSNR (biru): menunjukkan tren penurunan kualitas gambar seiring meningkatnya kompresi. PSNR turun dari sekitar 48.21 dB pada QF=90 hingga 28.53 dB pada QF=10.
- Grafik BER (merah): menunjukkan nilai BER terhadap threshold (garis putus-putus 0.3). BER secara konsisten berada di atas threshold pada semua nilai QF, dengan puncak tertinggi pada QF=10 (BER=0.92).
- Grafik NCC (hijau): menunjukkan nilai NCC terhadap threshold (garis putus-putus 0.5). NCC berada sangat jauh di bawah threshold pada semua nilai QF, mengkonfirmasi bahwa watermark tidak dapat diekstrak.



Grafik PSNR (kiri), BER (tengah), dan NCC (kanan) terhadap Quality Factor JPEG. Garis putus-putus menunjukkan threshold keberhasilan ekstraksi watermark

## 5.9 Grid Watermark Ter-Ekstrak per Quality Factor

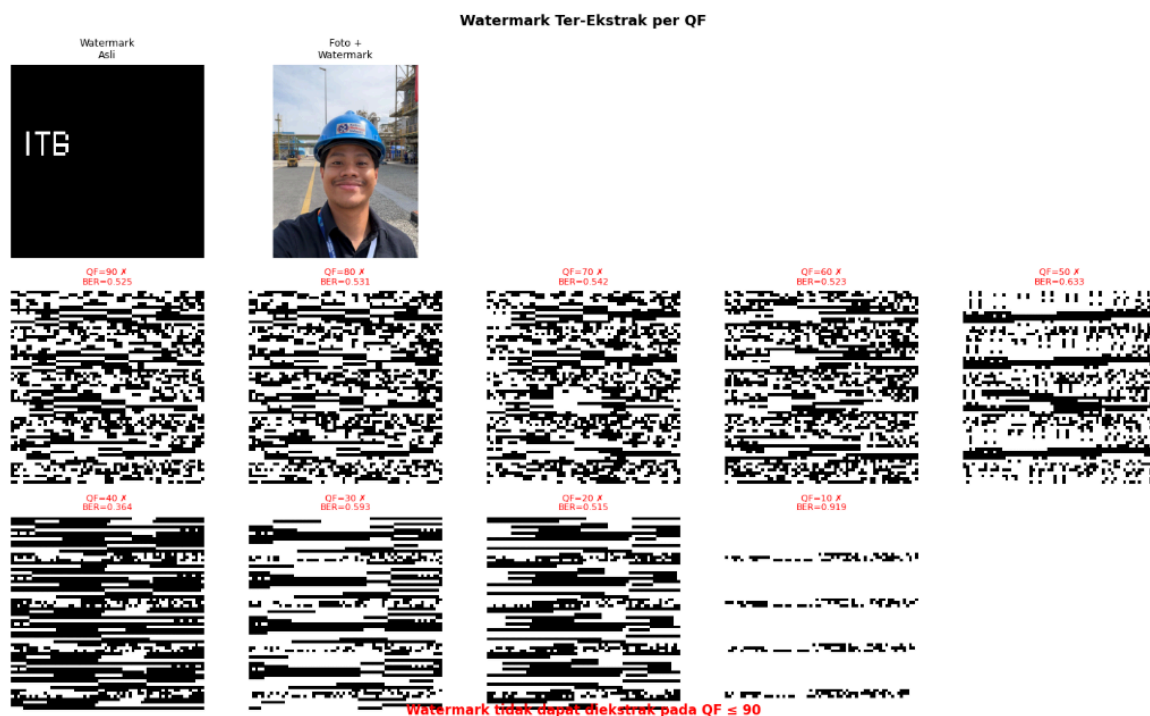
Visualisasi grid menampilkan perbandingan antara watermark asli dengan watermark hasil ekstraksi dari setiap versi gambar yang telah dikompres. Grid ini memberikan gambaran visual yang intuitif tentang seberapa parah degradasi watermark akibat kompresi JPEG.

Susunan grid adalah sebagai berikut:

- Baris pertama: (1) Watermark asli 'ITB' dan (2) Foto host yang telah disisipi watermark
- Baris kedua dan seterusnya: Watermark hasil ekstraksi untuk setiap nilai QF (dari QF=90 hingga QF=10), masing-masing dilengkapi label status (X) dan nilai BER

Dari visualisasi grid terlihat bahwa seluruh hasil ekstraksi watermark dari semua nilai QF tampak sebagai derau acak (noise) hitam-putih yang tidak membentuk pola teks

'ITB' yang dapat dikenali. Hal ini secara visual mengkonfirmasi hasil numerik yang telah diperoleh pada langkah sebelumnya.



## 5.10 Kesimpulan Eksperimen (Output Notebook)

Sel terakhir notebook menampilkan rangkuman hasil evaluasi secara terstruktur, mencakup metode embedding, ukuran watermark, hasil per QF, serta kesimpulan mengenai nilai QF threshold.

Output sel kesimpulan menampilkan:

- Metode embedding: LSB (channel Biru)
- Ukuran watermark:  $64 \times 64 \text{ bit} = 4.096 \text{ bit total}$
- QF yang diuji: [90, 80, 70, 60, 50, 40, 30, 20, 10]
- Hasil per QF: PSNR, BER, NCC, dan status ekstraksi untuk setiap nilai QF
- Kesimpulan: Watermark TIDAK dapat diekstrak pada  $QF \leq 90$



## 10. Kesimpulan

```
print('=' * 55)
print('KESIMPULAN')
print('=' * 55)
print(f'Metode embedding : LSB (channel Biru)')
print(f'Ukuran watermark : {WM_SIZE[0]} x {WM_SIZE[1]} bit = {WM_SIZE[0]*WM_SIZE[1]} bit total')
print(f'QF yang diuji : {QUALITY_FACTORS}')
print()
for r in results:
    st = '✓ Dapat diekstrak' if r['can_extract'] else 'X Tidak dapat diekstrak'
    psnr_str = f"{r['psnr']:.2f} dB" if r['psnr'] != float('inf') else '∞'
    print(f" QF={r['qf']:>3} PSNR={psnr_str:>10} BER={r['ber']:.4f} NCC={r['ncc']:.4f} → {st}")

print()
if threshold_qf:
    print(f'🔴 Watermark TIDAK dapat diekstrak pada QF ≤ {threshold_qf}')
    print(f' (LSB sangat rentan terhadap kompresi JPEG, karena')
    print(f' kompresi lossy mengubah bit-bit LSB channel warna)')
else:
    print(f'🟢 Watermark masih dapat diekstrak pada semua QF yang diuji.')
print('=' * 55)
```

```
*** =====
KESIMPULAN
=====
Metode embedding : LSB (channel Biru)
Ukuran watermark : 64 x 64 bit = 4096 bit total
QF yang diuji : [90, 80, 70, 60, 50, 40, 30, 20, 10]

QF= 90 PSNR= 48.21 dB BER=0.5254 NCC=-0.0099 → X Tidak dapat diekstrak
QF= 80 PSNR= 45.61 dB BER=0.5310 NCC=-0.0079 → X Tidak dapat diekstrak
QF= 70 PSNR= 44.00 dB BER=0.5415 NCC=-0.0132 → X Tidak dapat diekstrak
QF= 60 PSNR= 38.52 dB BER=0.5227 NCC=-0.0334 → X Tidak dapat diekstrak
QF= 50 PSNR= 35.08 dB BER=0.6328 NCC=-0.0428 → X Tidak dapat diekstrak
QF= 40 PSNR= 34.68 dB BER=0.3635 NCC=-0.0387 → X Tidak dapat diekstrak
QF= 30 PSNR= 33.68 dB BER=0.5925 NCC=-0.0002 → X Tidak dapat diekstrak
QF= 20 PSNR= 31.75 dB BER=0.5149 NCC=-0.0112 → X Tidak dapat diekstrak
QF= 10 PSNR= 28.53 dB BER=0.9187 NCC=-0.0268 → X Tidak dapat diekstrak

🔴 Watermark TIDAK dapat diekstrak pada QF ≤ 90
(LSB sangat rentan terhadap kompresi JPEG, karena
kompresi lossy mengubah bit-bit LSB channel warna)
=====
```

*Output sel kesimpulan yang menampilkan rangkuman hasil evaluasi watermarking*

Berdasarkan hasil implementasi dan evaluasi yang telah dilakukan, dapat disimpulkan sebagai berikut:

1. Implementasi watermarking menggunakan metode LSB (Least Significant Bit) berhasil dilakukan pada foto wajah pribadi berukuran 1280 x 959 piksel. Watermark biner berupa teks 'ITB' berukuran 64 x 64 piksel berhasil disisipkan ke dalam channel Biru gambar dengan nilai PSNR embedding sebesar 80.35 dB, yang berarti perubahan secara visual tidak terdeteksi oleh mata manusia.

2. Metode LSB terbukti sangat tidak tahan (fragile) terhadap kompresi JPEG. Pada semua nilai QF yang diuji (90, 80, 70, 60, 50, 40, 30, 20, 10), watermark tidak berhasil diekstrak kembali. Hal ini ditunjukkan oleh nilai BER yang selalu melebihi threshold 0.3 (berkisar 0.36–0.92) dan nilai NCC yang jauh di bawah threshold 0.5 (berkisar -0.04 hingga 0.04).
3. Nilai QF yang membuat watermark tidak dapat diekstrak adalah  $QF \leq 90$ , artinya bahkan pada kualitas kompresi JPEG tertinggi yang diuji sekalipun ( $QF=90$ ), watermark sudah tidak dapat diekstrak. Ini disebabkan karena proses kuantisasi DCT pada kompresi JPEG mengubah nilai piksel secara tidak terprediksi, sehingga merusak informasi yang tersimpan pada LSB.
4. Semakin rendah nilai QF (semakin tinggi tingkat kompresi), semakin rendah kualitas gambar (PSNR semakin kecil) dan semakin parah kerusakan watermark (BER mendekati 1). Penurunan PSNR dari 48.21 dB ( $QF=90$ ) ke 28.53 dB ( $QF=10$ ) menunjukkan korelasi langsung antara tingkat kompresi dan kualitas gambar.
5. Untuk aplikasi yang memerlukan ketahanan watermark terhadap kompresi JPEG, metode LSB tidak direkomendasikan. Metode watermarking berbasis domain frekuensi seperti DCT watermarking lebih sesuai digunakan karena menyisipkan informasi pada koefisien yang tidak sepenuhnya dihilangkan oleh proses kuantisasi JPEG.

## Referensi

Cox, I., Miller, M., Bloom, J., Fridrich, J., & Kalker, T. (2007). Digital watermarking and steganography (2nd ed.). Morgan Kaufmann Publishers.

Gonzalez, R. C., & Woods, R. E. (2018). Digital image processing (4th ed.). Pearson Education.

Hartung, F., & Kutter, M. (1999). Multimedia watermarking techniques. *Proceedings of the IEEE*, 87(7), 1079–1107.

Wallace, G. K. (1992). The JPEG still picture compression standard. *IEEE Transactions on Consumer Electronics*, 38(1), xviii–xxxiv.